

MODELLING PROGRAMS

MODELLING PROGRAMS

- **Creating a model** for the embedded system provides a **time-saving and cost-effective** approach to the development of **dynamic control systems**, based on a single model **maintained** in a tightly **integrated software** suite.
- It refers to a **visual method for** designing **complex control systems**

MODELLING PROGRAMS

- Modelling processes are used for **software analysis and design before software implementation.**
- A software analysis and design helps
 - A description of **the system requirement**
 - A description of **how system works**
 - **Shows system validation**
- Graphical modelling languages use a diagram technique with named symbols that represent concepts and lines that connect the symbols and represent relationships and various other graphical notation to represent constraints.

Computational Models in Embedded Design

- The commonly used computational models in embedded system design are:
 - Data Flow Graph Model
 - Control Data Flow Graph Model
 - State Machine Model
 - Sequential Program Model
 - Concurrent/Communicating Process Model
 - Object-Oriented Model

DATA FLOW MODEL

- A **program is modelled as handling the input data streams and creating output data streams.**
- A data-flow means that a program flow such that specifically the **data only determines all the program execution steps** and program flows are determined specifically only by the data.

DATA FLOW MODEL

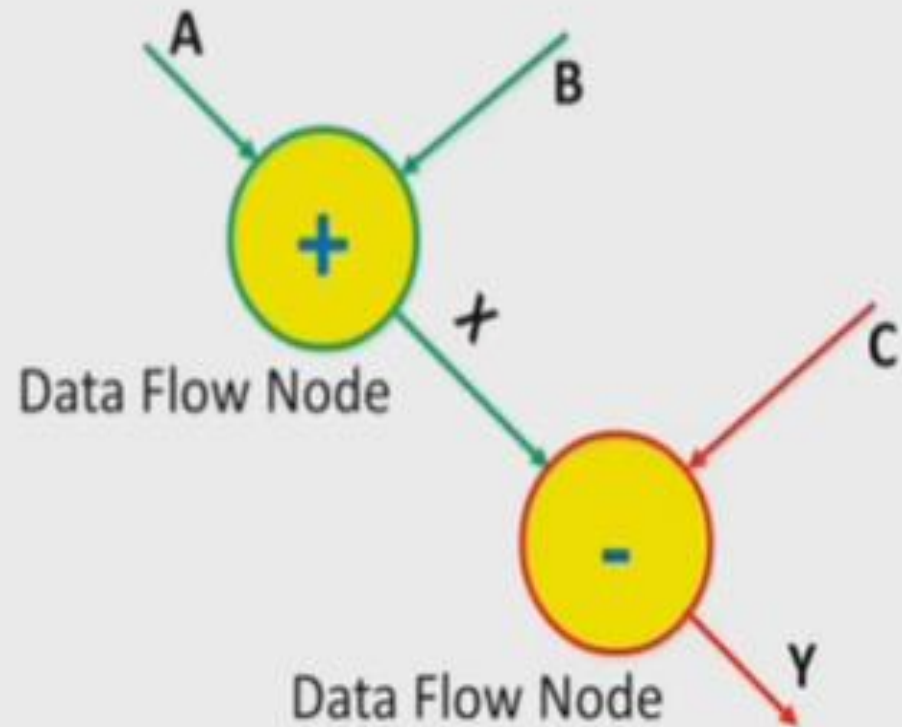
- **Programmer predetermines the data inputs** and designs the programming steps to **generate the data output.**
- A **set of data-input** generates a **set of data output** and **another set** of input will generate another output

What is Data Flow Graph Model

- ☐ Data driven model where program execution is driven by the data
- ☐ It translate the requirement in the data flow
- ☐ There are two types of DFG:
 - **A Cyclic DFG:**
No multiple values, no feedback, no events
 - **Non Cyclic DFG:**
System with feedback and events
- ☐ It is visual model where operations on the data is represented by using a circle
- ☐ Data flow is represented using arrows

Example of Data Flow Graph Model

- Inward arrow to circle ----- INPUT DATA
- Outward arrow to circle ----- OUTPUT DATA
- Example $X = A+B$ and $Y = X-C$

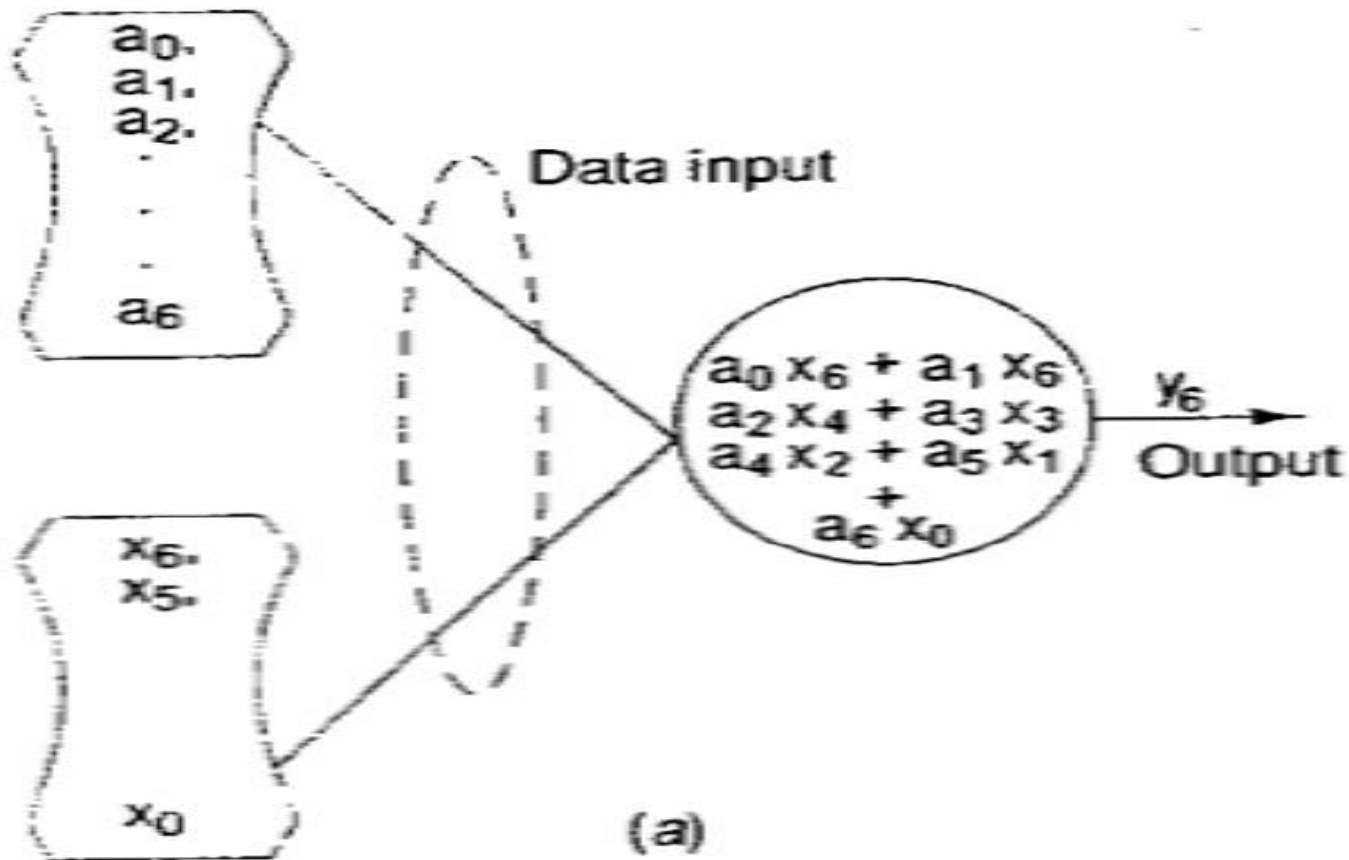


Example of Data Flow graph (DFG)

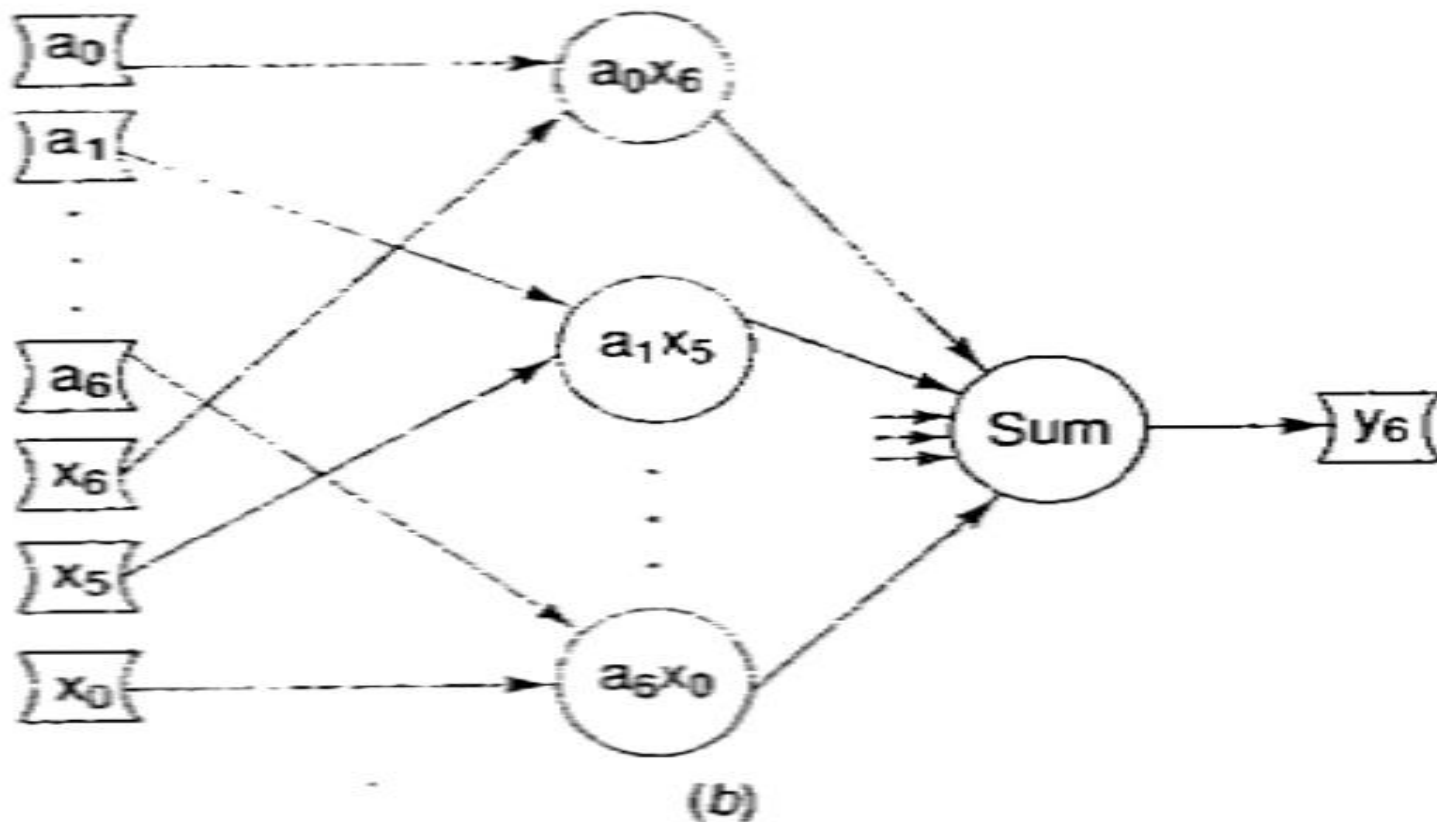
- A program for finding an **average of grades in various subjects** will have the **data inputs of grades and data output of the average.**
- Program **executes a function** to generate the appropriate output.
- **Name it as AVG FINDING**

- An **arrow directed towards** the circle **represents** the **data input** (or set of inputs) and an arrow **originating** from the **circle** represents a **data output (or a set of outputs)**.
- Data input along an **input edge is considered as token**
- An **input edge** has **at least one token**.

DFD-FINITE IMPULSE SEQUENCE(FIR)-AVG FUNCTION



DFD-FINITE IMPULSE SEQUENCE(FIR)-AVG FUNCTION (6 INP/6 OUT)



ADVANTAGES of DFG

- Data flow graph models help in a **simple code design.**
- which the **program mostly breaks into Number of DFGs.**
- A DFG models a fundamental program element having an **independent path.**

DISADVANTAGE OF DFG

- A DFG gives that unit of a system, **which has no control conditions** and thus a **single path for the program flow.**
- **Delay is Very high for complex applications**
- **Execution time is also high**

Control Data Flow Graph (CDFG) Modelling

- A **control flow** means that specifically **only** the **program determines all program execution steps and the flow of a program**
- The **programmer predetermines these steps**
- The steps may have **loops or condition statements in-between.**

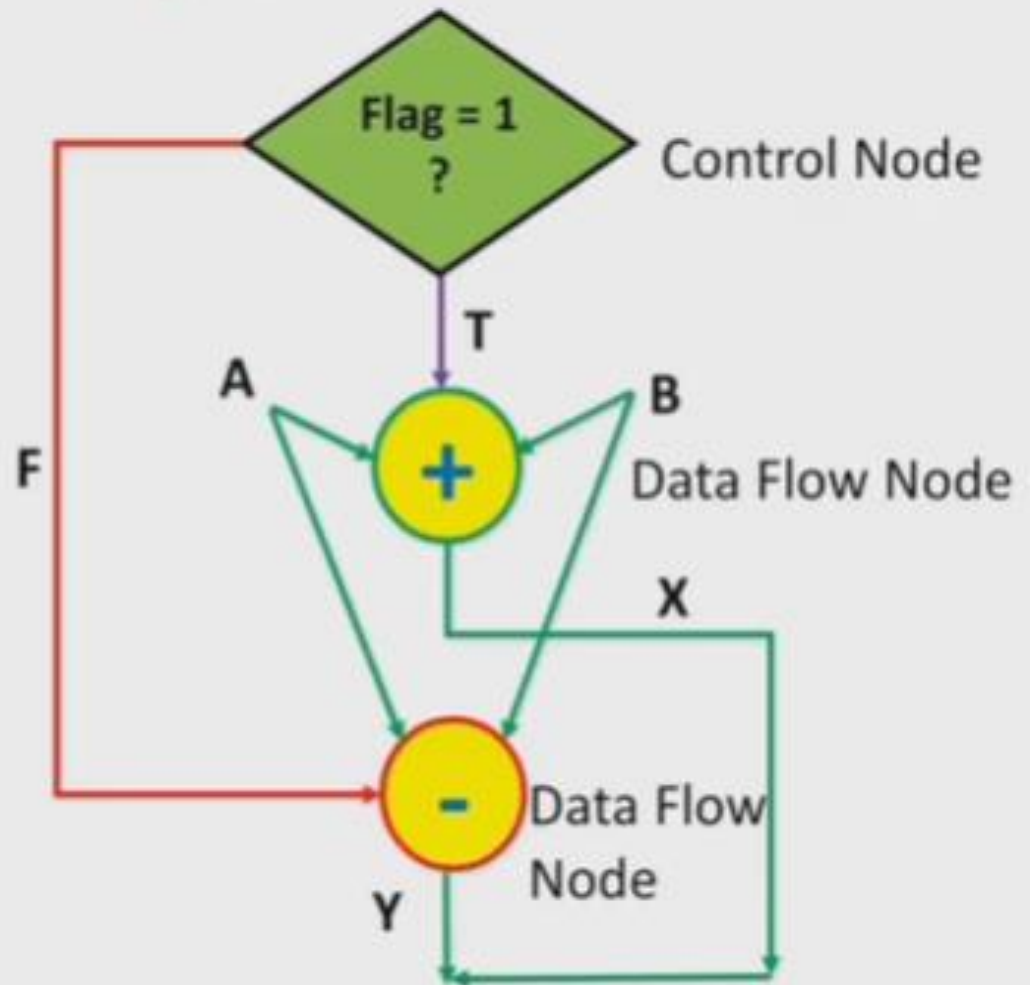
- Data that is input generate the **data output after a control data-flow** as per **controlling conditions**.
- Output(s) and **paths taken** after the steps **depends** on the **control statements** for various decisions in a process.

What is Conditional Data Flow Graph Model

- o Contains both data and control operations
- o Dataflow graph is elements and conditionals are decision makers
- o This mode has both data nodes and decision nodes whereas DFG only has data nodes
- o Example. Digital Still Camera
- Operation --- Circle
- Data Flow --- Arrow
- Control Node --- Diamond

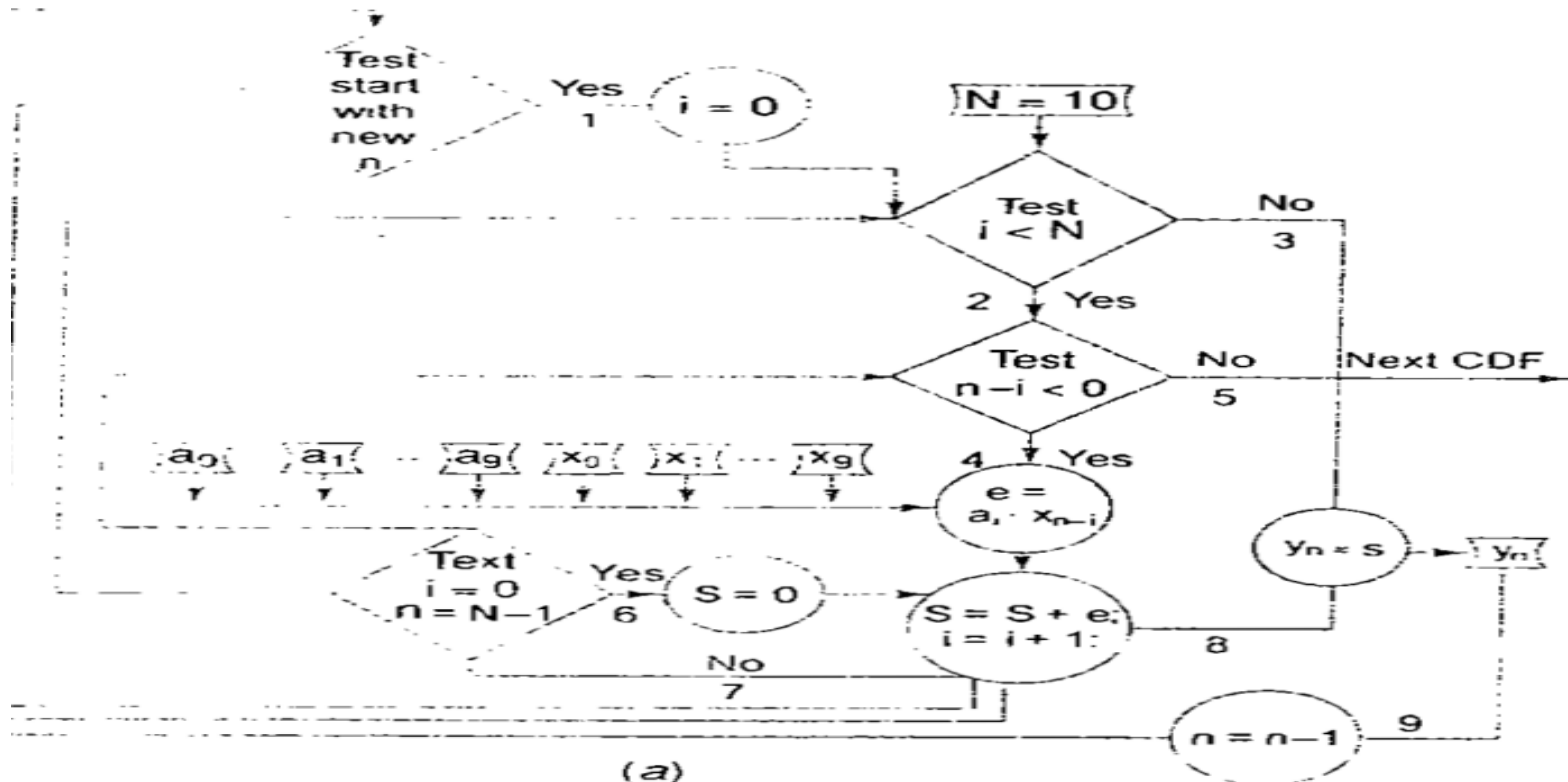
Example of Conditional Data Flow Graph Model

□ If **Flag = 1**, $X = A+B$ else $Y = A-B$



- CDFG model, there is a diagram, which graphically represents the conditions and the **program flow** along a **condition dependent path**.
- **Effect of events** among the processes and shows **which processes are activated on each specific event**.

Example-Control Data Flow Graph (CDFG) Modelling



ADVANTAGES - (CDFG) Modelling

- a **variable** value **changing above a limit or below a limit or falling** within a **range** is also like an **event** that activates a certain process.
- Guides us how software to be **tested for each path starting from a decision node**
- Helps in **analysing the program** in terms of **complexity**

Control Data Flow Graph/Diagram (CDFG) Model (continued)

- A real world example for modelling the embedded application using CDFG is capturing and saving of the image to a format set by the user in a digital still camera.
- Here everything is data driven.
 - Analog Front End converts the CCD sensor generated analog signal to Digital Signal
 - The data from ADC is stored to a frame buffer for the use of a media processor which performs various operations like, auto correction, white balance adjusting, etc.
- The decision on, in which format the image is stored (formats like JPEG, TIFF, BMP, etc.) is controlled by the camera settings, configured by the user.

FSM Model

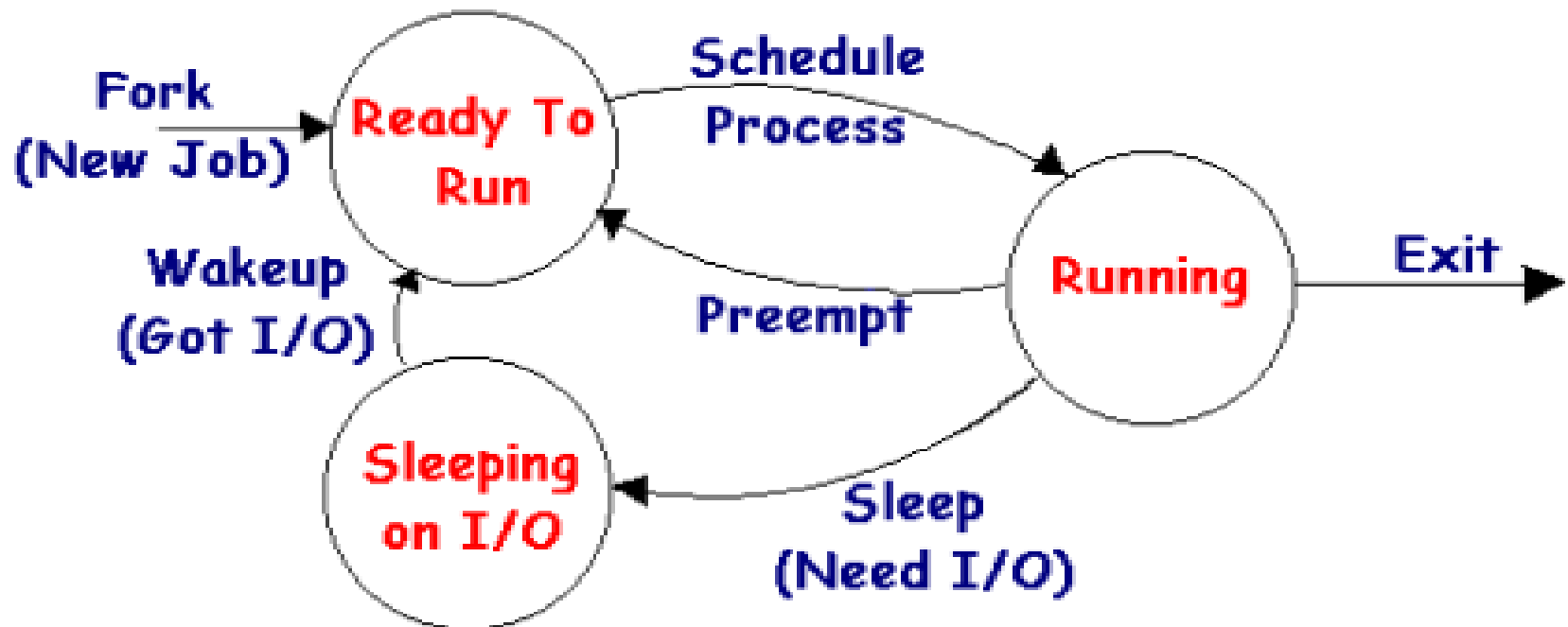
What Is State Machine Model

- ❑ This is reactive or event driven model
- ❑ Or embedded system whose processing behavior are dependent on state transition
- ❑ It is represented as “**States, Events, Actions & Transitions**”
- **State:** It is representation of current situation
- **Event:** It is representation of input to the state
- **Action:** It is representation of activity to be performed by state machine
- **Transition:** It is movement from one state to another

A STATE MACHINE AS:

- A set of input events
- A set of output events
- A set of states
- A function that maps **states and input to output**
- A function that maps **states and inputs to states** (which is called a state transition function)

Example: Process State diagram

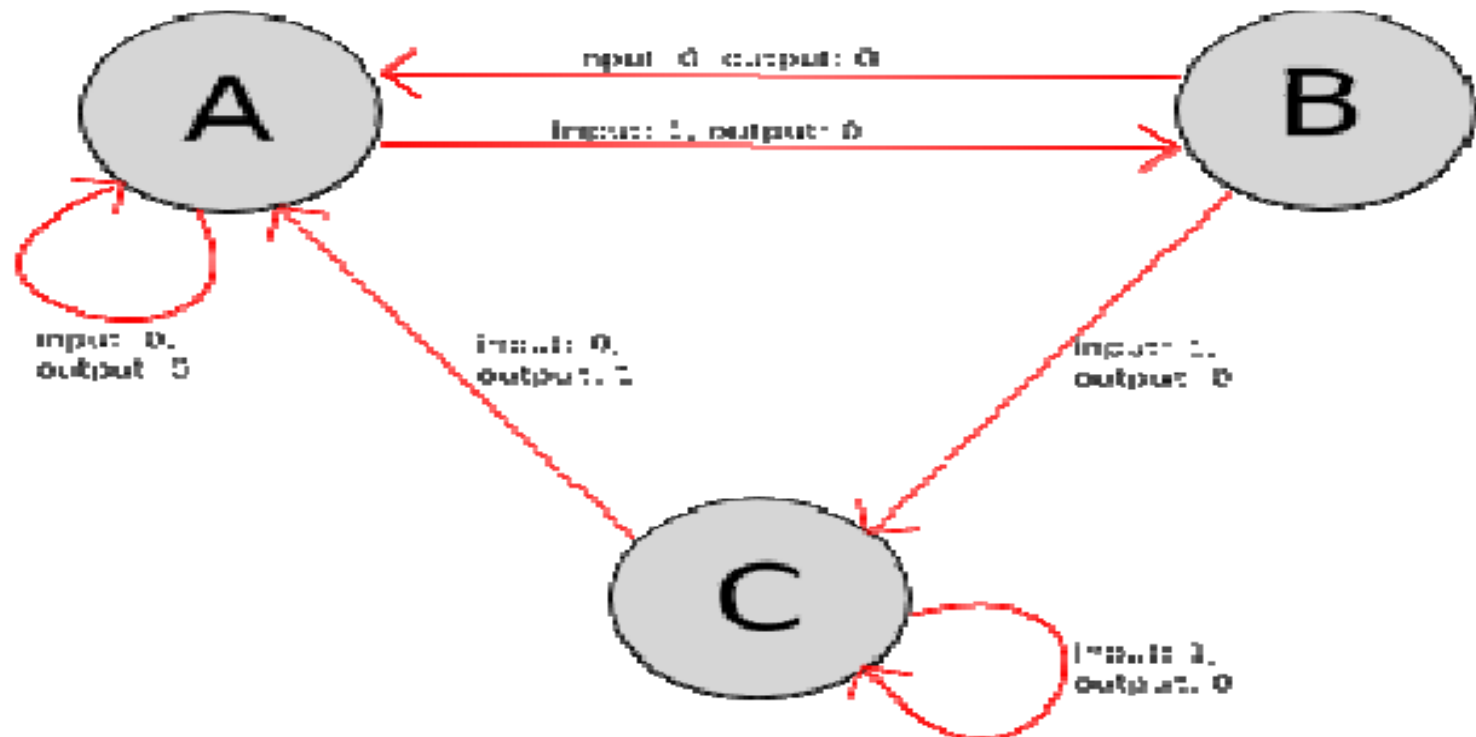


Finite States Machine(FSM) Model

- A finite state machine is one that has a **limited or finite number of possible states**.
- A finite state machine can be used both as a **development tool for approaching and solving problems** and as a formal way of describing the solution for later developers and system maintainers.
- There are a number of ways to show state machines, from **simple tables** through **graphically animated illustrations**.

Example

- The formal description of this state machine is the following: States:
- A: if input == 1 then { output 0, state = B } else { output 0, state = A }
- B: if input == 1 then { output 0, state = C } else { output 0, state = A }
- C: if input == 1 then { output 0, state = C } else { output 1, state = A }
- The initial state is A.
- The final state is C



Finite State Machine (FSM) Model

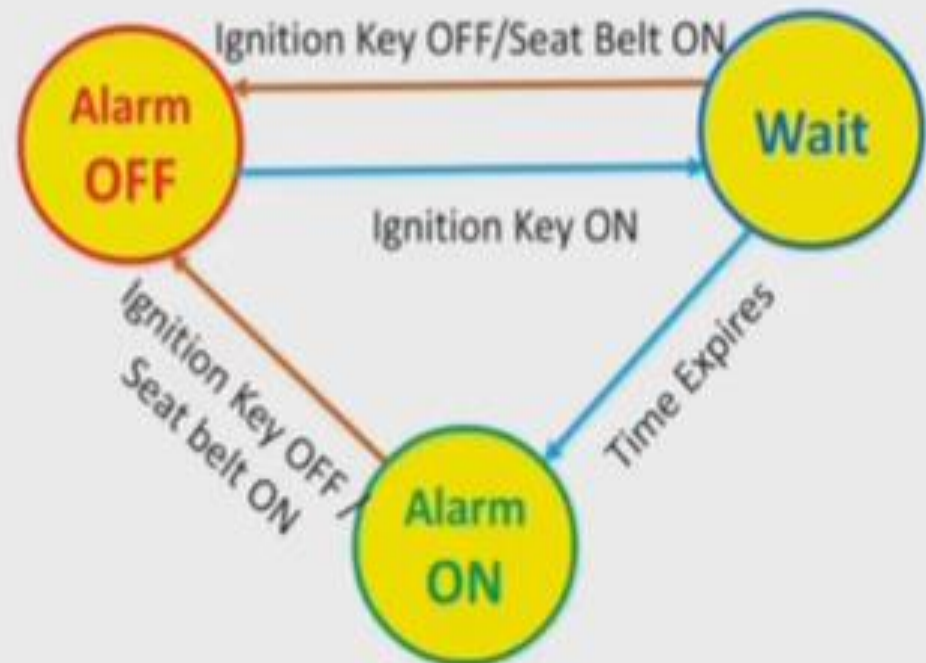
- A Finite State Machine (FSM) model is one in which the number of states are finite.
 - The system is described using a finite number of possible states.
- As an example, let us consider the design of an embedded system for driver/passenger 'Seat Belt Warning' in an automotive using the FSM model.
- The system requirements are captured as.
 1. When the vehicle ignition is turned on and the seat belt is not fastened within 10 seconds of ignition ON, the system generates an alarm signal for 5 seconds.
 2. The Alarm is turned off when the alarm time (5 seconds) expires or if the driver/passenger fastens the belt or if the ignition switch is turned off, whichever happens first.

Finite State Machine (FSM) Model (continued)

- Here the states are
 - 'Alarm Off'_I
 - 'Waiting'
 - 'Alarm On'
- The events are
 - 'Ignition Key ON'
 - 'Ignition Key OFF'
 - 'Timer Expire'
 - 'Alarm Time Expire'
 - 'Seat Belt ON'
- Using the FSM, the system requirements can be modeled as given in figure.

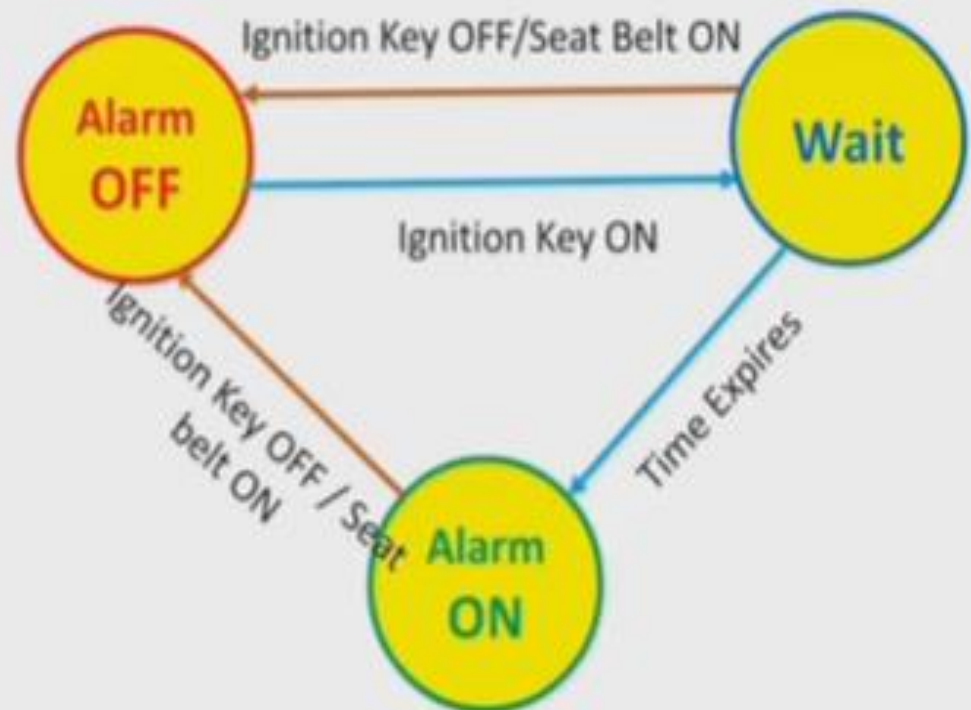
What Is Finite State Machine Model

- ❑ It is one in which number of states are finite
- ❑ Example: Embedded system for driver seat belt warning in automotive

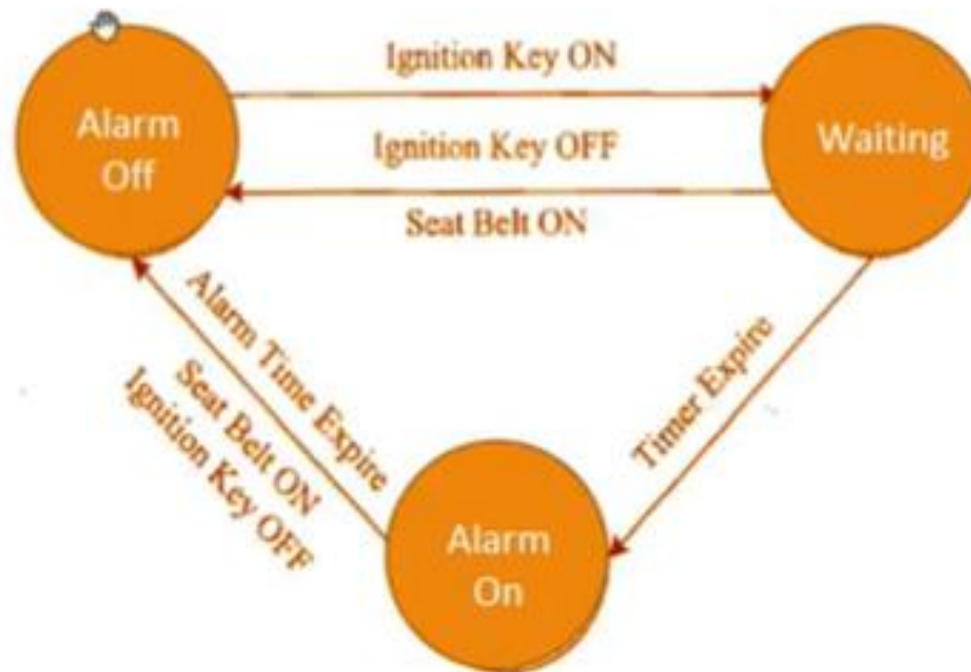


What Is Finite State Machine Model

- Here we can decide the **states** are Alarm Off, Waiting, Alarm ON and **events** are Ignition Key ON, Ignition Key OFF, Waiting Time Expires and Seat Belt ON using FMS



Finite State Machine (FSM) Model (continued)



FSM Model for Automatic Seat Belt Warning System

Finite State Machine (FSM) Model (continued)

- The 'Ignition Key ON' event triggers the 10 second timer and transitions the state to 'Waiting'.
- If a 'Seat Belt ON' or 'Ignition Key OFF' event occurs during the wait state, the state transitions into 'Alarm Off'.
- When the wait timer expires in the waiting state, the event 'Timer Expire' is generated and it transitions the state to 'Alarm On' from the 'Waiting' state.
- The 'Alarm On' state continues until a 'Seat Belt ON' or 'Ignition Key OFF' event or 'Alarm Time Expire' event, whichever occurs first.
- The occurrence of any of these events transitions the state to 'Alarm Off'.
- The wait state is implemented using a timer.
 - The timer also has certain set of states and events for state transitions.
 - Using the FSM model, the timer can be modelled as shown in the figure.

FSM Model - Example 1

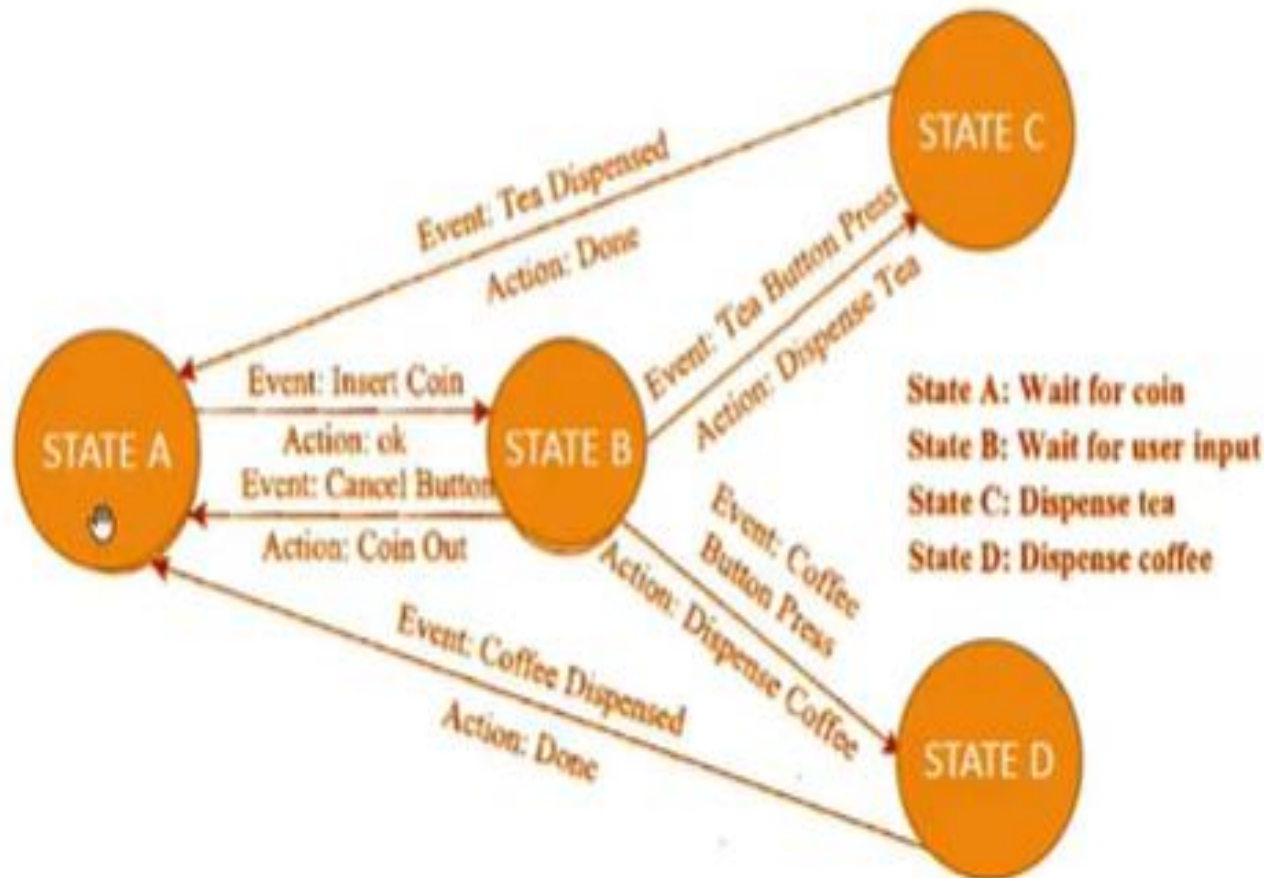
Design an automatic tea/coffee vending machine based on FSM model for the following requirement.

- The tea/coffee vending is initiated by user inserting a 5 rupee coin.
- After inserting the coin, the user can either select 'Coffee' or 'Tea' or press 'Cancel' to cancel the order and take back the coin.

Solution

- The FSM Model contains four states namely,
 - 'Wait for coin'
 - 'Wait for User Input'
 - 'Dispense Tea'
 - 'Dispense Coffee'

FSM Model - Example 1 (continued)



FSM Model for Automatic Tea/Coffee Vending Machine

FSM Model - Example 1 (continued)

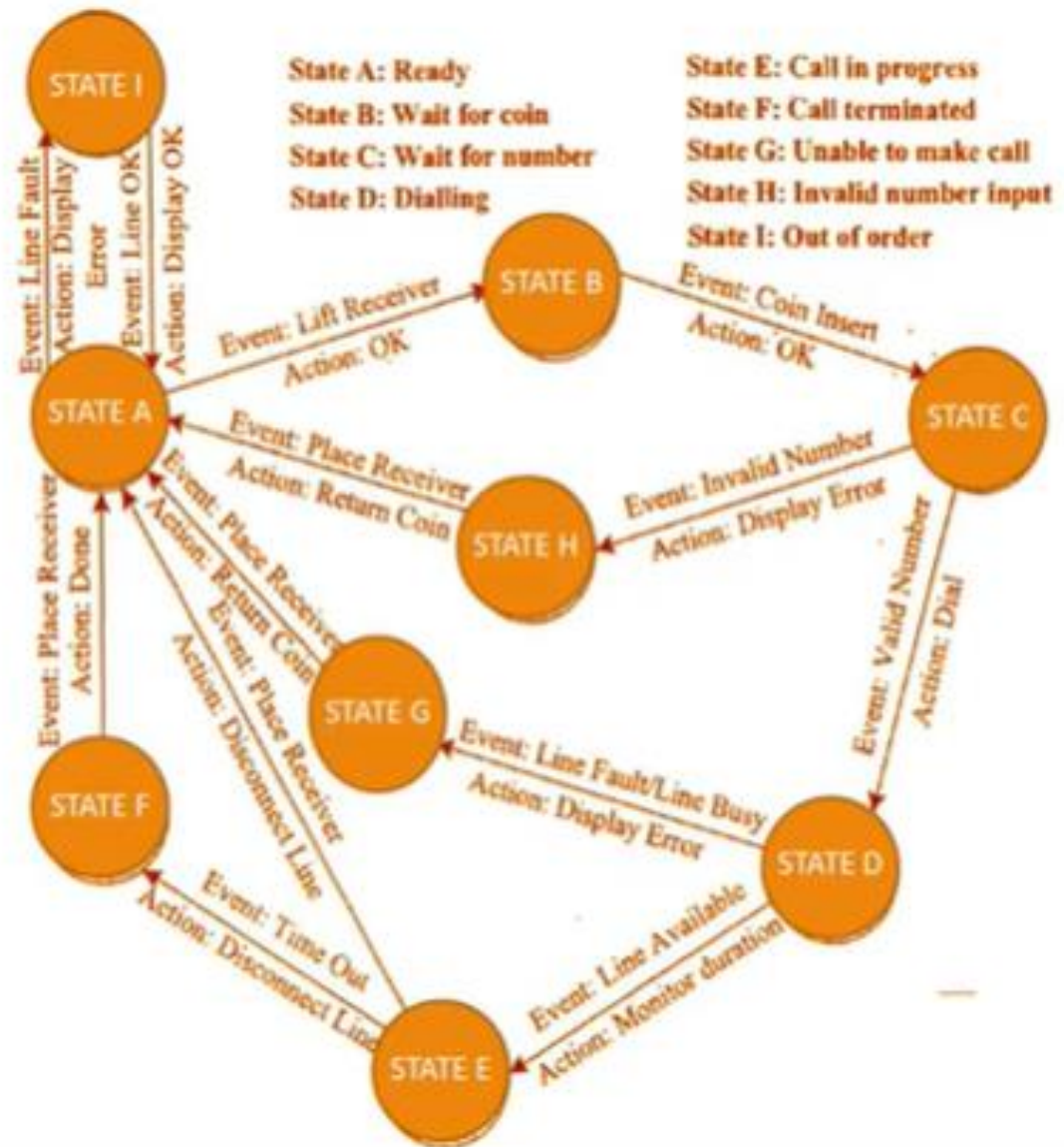
- The event 'Insert Coin' (5 rupee coin insertion), transitions the state to 'Wait for User Input'.
- The system stays in this state until a user input is received from the buttons 'Cancel', 'Tea' or 'Coffee' (Tea and Coffee are the drink select button).
- If the event triggered in 'Wait State' is 'Cancel' button press, the coin is pushed out and the state transitions to 'Wait for Coin'.
- If the event received in the 'Wait State' is either 'Tea' button press, or 'Coffee' button press, the state changes to 'Dispense Tea' and 'Dispense Coffee' respectively.
- Once the coffee/tea vending is over, the respective states transition back to the 'Wait for Coin' state.

FSM Model - Example 2

Design a coin operated public telephone unit based on FSM model for the following requirements.

1. The calling process is initiated by lifting the receiver (off-hook) of the telephone unit.
2. After lifting the phone the user needs to insert a 1 rupee coin to make the call.
3. If the line is busy, the coin is returned on placing the receiver back on the hook (on-hook).
4. If the line is through, the user is allowed to talk till 60 seconds and at the end of 45th second, prompt for inserting another 1 rupee coin for continuing the call is initiated.
5. If the user doesn't insert another 1 rupee coin, the call is terminated on completing the 60 seconds time slot.
6. The system is ready to accept new call request when the receiver is placed back on the hook (on-hook).
7. The system goes to the 'Out of Order' state when there is a line fault.

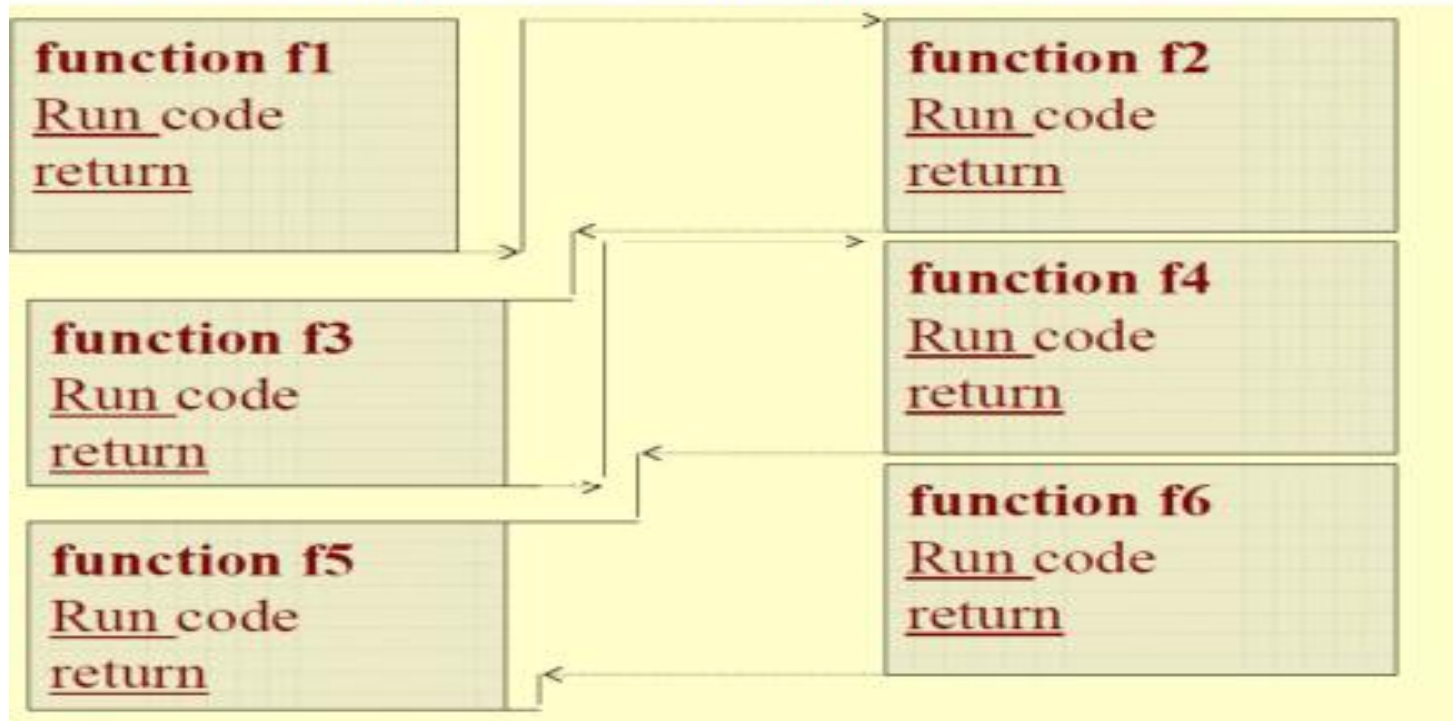
FSM Model for Coin Operated Telephone System 📞



SEQUENTIAL PROGRAM MODEL

- Use of multiple function calls sequentially

Programming model of Six sequential function calls



Example Sequential Program

Model=VM

- Run function `get_user_input ( )` for obtaining input for choice of chocolate from child.
- Run function `read_coins ( )` for reading the coins inserted into `VendingMachine(VM)` for cost of chocolate.
- Run function `deliver_chocolate()` for delivering the chocolate.
- Run function `display_thanks()` for displaying 'Collect the nice chocolate. Visit again!'

Sequential Program Model=VM

